Auth API Reference

Sign up, sign in and session endpoints · version 1.0

Base URL	`http://localhost:3000/api` (via the web app: `http://localhost:5173/api`)
Format	JSON requests and responses (`Content-Type: application/json`)
Auth	Session cookie `access_token` (httpOnly), or `Authorization: Bearer <token>`

Overview

Endpoint	Purpose	Auth
`POST /auth/signup`	Create an account and start a session	Public
`POST /auth/signin`	Sign in and start a session	Public
`POST /auth/logout`	End the session	Public
`GET /auth/me`	Return the signed-in user (the protected endpoint)	Session
`GET /health`	Liveness and database readiness	Public

Authentication & sessions

- Sign-up and sign-in create a **session record in MongoDB** and return a signed JWT (HS256) that carries only the user id (`sub`) and session id (`sid`).
- The JWT is set as the `access_token` cookie: **httpOnly**, `SameSite=Lax`, and `Secure` in production. Browsers send it automatically.
- Non-browser clients (curl, Postman) can send the same token as `Authorization: Bearer <token>`.
- On every protected request the server checks the signature, the expiry **and** that the session still exists and belongs to that user.
- **Logout deletes the session**, so a copied token stops working immediately. Other sessions of the same user are not affected.
- Sessions last 1 hour by default (`JWT_EXPIRES_IN_SECONDS`) and expired sessions are removed automatically.
- There is no refresh token: after the session expires the user signs in again.

Validation rules

Field	Rule
`email`	Valid email format, at most 254 characters. Trimmed and stored in lowercase, so `Amr@Example.com` and `amr@example.com` are the same account.
`name`	At least 3 and at most 50 characters (measured after trimming).
`password`	8 to 72 characters, with at least one letter, one number and one special character. (72 is bcrypt's limit.)

Unknown fields in the body are rejected with `400` rather than ignored.

Rate limiting

Scope	Limit
All endpoints (per client IP)	100 requests per minute
`POST /auth/signup`, `POST /auth/signin`	10 requests per minute
`GET /health`	Not limited

Exceeding a limit returns `429 Too Many Requests`.

Trying the API from the command line

The examples in this document use `curl`. `-c cookies.txt` saves the session cookie and `-b cookies.txt` sends it back, so you can sign in once and then call protected endpoints.

Windows PowerShell: `curl` is an alias for a different command. Use `curl.exe`, and put the JSON body in a file (`-d @body.json`) to avoid quoting problems.

Endpoints

POST `/api/auth/signup` — Public

Create an account and start a session

Rate limit: 10 per minute per IP

Request body (JSON)

Field	Type	Required	Rules	Example
`email`	string	Yes	Valid email, max 254 chars; stored lowercase	`amr@example.com`
`name`	string	Yes	3 to 50 characters	`Amr Saeed`
`password`	string	Yes	8 to 72 chars; at least one letter, one number, one special character	`Str0ng!pass`

Responses

Status	Meaning
201	Account created. Session cookie set. Body contains the public user.
400	Validation failed. `message` is an array listing **every** broken rule.
409	An account with this email already exists.
429	Too many attempts. Try again later.

Example request

```
curl -i -c cookies.txt -X POST http://localhost:3000/api/auth/signup \
  -H "Content-Type: application/json" \
  -d '{"email":"amr@example.com","name":"Amr Saeed","password":"Str0ng!pass"}'
```

Example response

```
HTTP/1.1 201 Created
Set-Cookie: access_token=<jwt>; Path=/; HttpOnly; SameSite=Lax

{
  "user": {
    "id": "665f1c2e9b1e8a0012a4b7c1",
    "name": "Amr Saeed",
    "email": "amr@example.com",
    "createdAt": "2026-10-31T10:15:00.000Z"
  }
}
```

POST `/api/auth/signin` — Public

Sign in and start a session

Rate limit: 10 per minute per IP

Wrong password and unknown email return the **same** 401 message, so the API does not reveal which emails are registered.

Request body (JSON)

Field	Type	Required	Rules	Example
`email`	string	Yes	Valid email format	`amr@example.com`
`password`	string	Yes	Not empty, at most 72 characters (complexity is not re-checked)	`Str0ng!pass`

Responses

Status	Meaning
200	Signed in. Session cookie set. Body contains the public user.
400	Validation failed (missing or malformed fields).
401	Invalid email or password.
429	Too many attempts. Try again later.

Example request

```
curl -i -c cookies.txt -X POST http://localhost:3000/api/auth/signin \
  -H "Content-Type: application/json" \
  -d '{"email":"amr@example.com","password":"Str0ng!pass"}'
```

Example response

```
HTTP/1.1 200 OK
Set-Cookie: access_token=<jwt>; Path=/; HttpOnly; SameSite=Lax

{
  "user": {
    "id": "665f1c2e9b1e8a0012a4b7c1",
    "name": "Amr Saeed",
    "email": "amr@example.com",
    "createdAt": "2026-10-31T10:15:00.000Z"
  }
}
```

GET /api/auth/me

Requires session

Protected: return the signed-in user

The protected endpoint. Send the session cookie, or a Bearer token.

Responses

Status	Meaning
200	The signed-in user.
401	No session cookie or token, the token is invalid or expired, or the session was ended (for example by logout).

Example request

```
curl -b cookies.txt http://localhost:3000/api/auth/me
```

Example response

```
{
  "id": "665f1c2e9b1e8a0012a4b7c1",
  "name": "Amr Saeed",
  "email": "amr@example.com",
  "createdAt": "2026-10-31T10:15:00.000Z"
}
```

POST /api/auth/logout

Public

End the session: delete it server-side and clear the cookie

Deletes the session on the server and clears the cookie. It is deliberately **not** behind the auth guard, so an expired or unknown token can still clear its cookie. It always answers `204`, so it cannot be used to probe whether a token is valid.

Responses

Status	Meaning
204	Session ended and cookie cleared. No body.

Example request

```
curl -i -b cookies.txt -c cookies.txt -X POST http://localhost:3000/api/auth/logout
```

Example response

```
HTTP/1.1 204 No Content
Set-Cookie: access_token=; Path=/; Expires=Thu, 01 Jan 1970 00:00:00 GMT; HttpOnly;
SameSite=Lax
```

GET /api/health

Public

Liveness + database readiness probe

Rate limit: not limited

Used by load balancers and container health checks.

Responses

Status	Meaning
200	The app is up and MongoDB is connected.
503	MongoDB is not connected.

Example request

```
curl http://localhost:3000/api/health
```

Example response

```
{ "status": "ok" }
```

Error format

Every error, from any endpoint, has the same shape:

```
{
  "statusCode": 400,
  "error": "Bad Request",
  "message": [
    "Name must be at least 3 characters",
    "Password must contain at least one number"
  ],
  "path": "/api/auth/signup",
  "timestamp": "2026-10-31T10:15:00.000Z",
  "requestId": "6f1c7d0e-3f0b-4c0a-9a53-2b1e0b6e7f10"
}
```

Field	Description
`statusCode`	HTTP status code.
`error`	Short name of the status.
`message`	A string, or an array of strings with one entry per failed validation rule.
`path`	The request path.
`timestamp`	When the error was produced (ISO 8601, UTC).
`requestId`	Also returned in the `x-request-id` response header. Quote it when reporting a problem; it matches the server log lines. Send your own `x-request-id` to have it reused.

Status codes used

Code	When
200 / 201 / 204	Success.
400	Validation failed, including unknown fields in the body.
401	Not signed in, or wrong credentials.
404	Unknown route.
409	Email already registered.
429	Rate limit exceeded.
500	Unexpected error. The response is a generic message; details are only in the server log.
503	Database not connected (health check).